

코딩테스트 체크리스트 — 서술형 모범 답안

목적: 단순 암기를 넘어 원리와 판단 근거를 명확히 하여 실전 대응력을 높임.

1단계: 메서드 암기 (핵심 원리)

06. ArrayList의 크기를 반환하는 메서드는? 왜 $O(1)$ 인가?

답안: 내부에 `size`라는 변수를 따로 관리하며, 요소가 추가/삭제될 때마다 이 값을 업데이트해두기 때문에 호출 시점에 전체를 세지 않아도 됩니다.

07. ArrayList에서 부분 리스트를 추출하는 메서드는? 반환값은 복사본인가 뷰인가?

답안: ****뷰(View)****입니다. 추출된 리스트를 수정하면 원본 리스트도 함께 수정되며, 반대의 경우도 마찬가지입니다.

08. ArrayList를 정렬하는 메서드는? 내부적으로 어떤 알고리즘을 쓰는가?

답안: Timsort (Insertion Sort와 Merge Sort를 결합한 하이브리드 알고리즘)를 사용합니다. 데이터의 정렬 상태에 따라 최적화된 성능을 냅니다.

13. Java에서 스택이 필요할 때 Stack 클래스 대신 뭘 쓰는가? 왜?

답안: ****Deque(ArrayDeque)****를 권장합니다. `Stack` 클래스는 멀티스레드 환경을 고려한 동기화 (`synchronized`) 처리가 되어 있어 단일 스레드인 코테 환경에서는 성능이 떨어지고, 상속 구조상 벡터의 성질을 가져 객체지향 원칙에도 어긋나기 때문입니다.

19. merge에서 사용하는 `::`는 무슨 문법인가?

답안: 메서드 참조(Method Reference) 문법입니다. 람다식을 더 간결하게 표현한 것으로, 여기서는 기존 값과 새 값을 어떻게 연산할지 정의된 메서드를 직접 연결합니다.

21. HashMap에서 값 존재 여부를 확인하는 메서드는? 왜 $O(n)$ 인가?

답안: 키는 해시 함수를 통해 위치를 즉시 찾을 수 있지만, 값은 정해진 위치가 없으므로 최악의 경우 모든 엔트리를 다 뒤져봐야 하기 때문입니다.

22. HashMap의 모든 키를 반환하는 메서드는? 반환 타입은? 왜 Set인가?

답안: Set입니다. Map의 키는 중복될 수 없는 고유한 값들의 집합이기 때문에 Set의 성질과 일치합니다.

23. HashMap의 모든 값을 반환하는 메서드는? 반환 타입은? 왜 Collection인가?

답안: Collection입니다. Map의 값은 중복될 수 있으므로 중복을 허용하지 않는 Set을 쓸 수 없고, 특정 순서를 보장하는 List라고 단정할 수도 없기 때문입니다.

25. HashMap에 `set()` 메서드가 없는 이유는?

답안: Map은 인덱스 기반의 자료구조가 아니며, 특정 키에 값을 넣는 행위 자체가 '이미 있으면 수정, 없으면 추가'라는 의미를 내포하고 있어 `put`이 그 역할을 대신합니다.

26. getOrDefault와 putIfAbsent의 차이는?

답안: `getOrDefault`는 키가 없으면 기본값만 반환하고 Map을 건드리지 않지만, `putIfAbsent`는 키가 없으면 기본값을 실제로 저장하고 null을 반환합니다.

27. merge와 getOrDefault+put 중 어떤 상황에서 어느 것을 쓰는가?

답안: 단순히 값을 읽어올 때 기본값이 필요한 거라면 `getOrDefault`를, 누적 합계(카운팅)처럼 기존 값과 새 값의 연산 결과를 다시 저장해야 할 때는 `merge`가 훨씬 효율적입니다.

28. HashMap의 순서가 필요하면 무엇을 쓰는가?

답안: `LinkedHashMap`(입력 순서) 또는 `TreeMap`(키 기준 정렬 순서)을 사용합니다.

34. Comparable과 Comparator의 차이는? 코테에서 어느 걸 쓰는가?

답안: `Comparable`은 객체 스스로가 가진 기본 정렬 기준을 정의할 때 쓰고, `Comparator`는 상황에 따라 기본 기준과 다른 방식으로 정렬하고 싶을 때 별도로 생성해서 씁니다. 코테에서는 보통 람다를 활용한 `Comparator`를 훨씬 많이 씁니다.

41. 스트림의 3가지 특성은?

답안:

- 원본 불변: 스트림은 데이터를 읽기만 할 뿐 원본을 수정하지 않습니다.
- 일회용: 한 번 소비된 스트림은 재사용할 수 없습니다.
- 지연 평가: 최종 연산이 호출되기 전까지는 중간 연산이 실제로 실행되지 않아 최적화가 가능합니다.

42. 중간 연산과 최종 연산의 차이는?

답안: 중간 연산은 스트림을 다시 반환하여 메서드 체이닝을 가능하게 하고, 최종 연산은 스트림을 닫으면서 결과를 도출(List 반환, 합계 계산 등)합니다.

47. splice와 slice의 차이를 한 문장으로 말해라

답안: `splice`는 원본 배열을 **직접 수정(삭제/삽입)**하고, `slice`는 원본은 그대로 둔 채 필요한 부분만 복사해서 새 배열을 만듭니다.

55. Object에서 키 존재 여부를 확인하는 2가지 방법은? 차이는?

답안: `key in obj` 방식과 `obj.hasOwnProperty(key)` 방식이 있습니다. `in`은 객체가 상속받은 프로토타입 체인까지 확인하고, `hasOwnProperty`는 객체 스스로가 가진 키만 확인합니다.

57. Object와 Map 중 어떤 상황에서 Map을 써야 하는가?

답안: 키의 추가/삭제가 빈번하거나, 키로 문자열 외의 타입(객체 등)을 써야 할 때, 혹은 데이터의 순서 보장이 필요할 때는 Map이 성능과 기능 면에서 유리합니다.

61. JS에서 sort()를 콜백 없이 호출하면 어떻게 되는가? 왜 위험한가?

답안: 모든 요소를 문자열로 변환한 뒤 사전순으로 정렬합니다. 예를 들어 `[10, 2]`를 정렬하면 `[10, 2]`가 나옵니다. ('1'이 '2'보다 앞서기 때문)

63. 콜백 반환값이 음수/0/양수일 때 각각 무슨 의미인가?

답안: 음수면 첫 번째 인자가 앞으로(오름차순), 양수면 두 번째 인자가 앞으로 옵니다. 0이면 순서를 바꾸지 않습니다.

64. sort()는 원본을 변경하는가? 원본을 보존하면서 정렬하려면?

답안: 원본을 변경합니다. 원본 보존이 필요하다면 스프레드 연산자 등으로 복사본을 만든 뒤 정렬해야 합니다 (`[...arr].sort()`).

68. 모든 요소를 하나의 값으로 합치는 메서드는? 초기값을 왜 항상 넣어야 하는가?

답안: 초기값을 넣지 않으면 배열의 첫 번째 요소가 초기값이 되는데, 빈 배열일 경우 에러가 발생하거나 원치 않는 타입(객체 배열에서 객체 자체가 연산에 쓰이는 등)으로 처리될 수 있어 안정성을 위해 필수입니다.

72. forEach의 반환값은? 왜 체이닝이 안 되는가?

답안: `forEach`는 단순히 각 요소를 순회하며 외부 로직을 실행할 뿐, `undefined`를 반환하기 때문에 결과값을 받아 다음 단계로 넘기는 체이닝이 불가능합니다.

73. map에서 중괄호를 쓸 때 흔히 하는 실수는?

답안: 화살표 함수에서 중괄호(`{}`)를 쓰면 명시적으로 `return`을 작성해야 합니다. 작성하지 않으면 `undefined`가 반환되어 맵핑 결과가 깨집니다.

2단계: 패턴 적용

76. JS에서 빈도 세기의 `(obj[key] || 0) + 1` 패턴을 설명하라

답안: 객체에 키가 있으면 그 값을 쓰고, 없으면(`undefined`면) 0을 기본값으로 사용하겠다는 단락 평가(Short-circuit evaluation) 패턴입니다.

78. "서로 다른 종류의 개수"를 구하라는 문제가 나왔다. HashMap vs HashSet 중 뭘 쓰는가? 판단 기준은?

답안: 단순히 "중복 제외 총 몇 개?"만 묻는다면 `HashSet`이 메모리 효율적입니다. 하지만 "가장 많이 나온 것"처럼 값에 대한 추가 정보가 필요하다면 `HashMap`으로 빈도를 기록해야 합니다.

80. 고유 개수만이 아니라 각 종류별 개수까지 필요하다면 어떻게 접근하는가?

답안: 위에서 언급한 빈도 세기(카운팅) 로직을 먼저 적용하여 Map을 만든 뒤, 그 Map의 `size`를 보거나 value들을 분석합니다.

81. 포켓몬 문제: N/2마리를 골라 최대 종류 수를 구하는 핵심 아이디어는?

답안: `Math.min(전체 종류 수, 가져갈 수 있는 마리 수 N/2)`가 정답입니다. 아무리 종류가 많아도 N/2마리밖에 못 가져가기 때문입니다.

82. 숫자 배열을 이어붙여서 가장 큰 수를 만들어라. 정렬 기준을 어떻게 정의하는가?

답안: 두 수를 앞뒤로 붙여본 결과($a + b$ vs $b + a$)를 비교하여 더 큰 숫자가 나오는 순서로 정렬합니다.

84. 내림차순 정렬 후 i 번째 값 * $(i+1)$ 의 최대값을 구하는 문제(로프 문제)의 접근법은?

답안: 로프를 무거운 순서대로 정렬(내림차순)한 뒤, 현재 로프의 무게에 지금까지 선택한 로프의 개수($i + 1$)를 곱해나가며 그 중 최댓값을 찾습니다.

85. Java에서 sort()는 원본을 변경하는가? Stream의 sorted()는?

답안: Collections.sort()나 Arrays.sort()는 원본을 변경하지만, Stream의 sorted()는 원본을 건드리지 않고 정렬된 새 스트림을 생성합니다.

86. ArrayList의 contains 대신 HashSet을 쓰는 이유는? 시간복잡도 차이는?

답안: ArrayList.contains()는 $O(N)$ 이지만, HashSet은 해시값을 이용해 $O(1)$ 에 찾습니다. 전화번호가 수만 개일 때 접두사를 일일이 찾으려면 효율성 차이가 매우 커집니다.

87. 접두사 검사에서 substring을 어떻게 활용하는가? 접미사는?

답안: 번호를 길이순으로 정렬한 뒤, 각 번호를 해시셋에 넣기 전에 해당 번호의 0번 인덱스부터 i 번째까지 자른 문자열들이 이미 셋에 존재하는지 확인합니다.

88. 존재 여부만 확인하면 HashSet, 부가 정보가 필요하면 HashMap — 이 기준을 설명하라

답안: "이게 있었나?"만 궁금하면 Set, "이게 몇 번 나왔지?" 또는 "이 사람의 점수가 몇 점이지?"처럼 연결된 정보가 필요하면 Map입니다.

89. 같은 종류끼리 2개를 고르는 조합 수 공식은?

답안: $n \times (n - 1) / 2$ 입니다.

90. 빈도를 센 뒤 entrySet(Java) / Object.entries(JS)로 순회하며 조합을 계산하는 흐름을 설명하라

답안: Map의 모든 엔트리를 순회하면서, 각 종류별로 선택 가능한 경우의 수를 누적해서 곱하거나 더하는 방식으로 전체 조합을 완성합니다.

91. 의상 조합 문제에서 "각 종류별 (개수+1)을 곱하고 -1"하는 이유를 설명하라

답안: 각 의상 종류에서 "하나를 입는 경우(n 개)"에 "아무것도 입지 않는 경우(1개)"를 더해 모두 곱한 뒤, 최소한 하나는 입어야 하므로 "모두 안 입는 경우(1개)"를 빼주는 것입니다.

92. 배열에서 조건에 맞는 요소만 골라 변환한 결과가 필요하다. for문 vs Stream/고차함수 중 어떤 기준으로 판단하는가?

답안: 단순 성능이 극도로 중요하다면 for문이 약간 빠를 수 있으나, 가독성과 유지보수, 그리고 실수 방지 면에서는 스트림/고차함수가 유리합니다. 로직이 복잡해질수록 스트림이 실수를 줄여줍니다.

96. 중복 제거 + 정렬이 필요할 때 Java와 JS 각각 어떻게 하는가?

답안: Java는 distinct().sorted(), JS는 [...new Set(arr)].sort() 패턴을 주로 사용합니다.

97. 연속 중복 제거 문제에서 ArrayList를 스택처럼 쓰는 흐름을 설명하라

답안: 리스트를 순회하며 **마지막에 넣은 요소(peek)**와 현재 넣으려는 요소가 같으면 넣지 않고 넘어가거나, 다를 때만 push 하는 방식으로 구현합니다.

98. 괄호 유효성 검사에서 스택을 어떻게 활용하는가? (JS push/pop 기준)

답안: 여는 괄호는 stack에 `push`하고, 닫는 괄호가 나오면 stack에서 `pop`하여 짝이 맞는지 확인합니다. 순회가 끝났을 때 stack이 비어있어야 유효합니다.

99. "두 수의 합이 target인 쌍 찾기" — 이중 반복문 $O(n^2)$ 과 HashMap $O(n)$ 중 어떤 걸 쓰는가? 데이터 크기에 따른 판단 기준은?

답안: 데이터가 1만 건 이상이라면 $O(n^2)$ 은 위험합니다. `target - 현재값`이 Map에 존재하는지 $O(1)$ 로 확인하는 `**HashMap 방식( $O(n)$ )`을 선택해야 합니다.

100. ArrayList의 `contains()`는 $O(n)$, HashSet의 `contains()`는 $O(1)$ 이다. 반복문 안에서 존재 여부를 확인해야 할 때 어떤 걸 써야 하는가? 전체 시간복잡도 차이는?

답안: 반드시 HashSet을 써야 합니다. 반복문(N번) 안에서 `contains`를 호출하면 전체 시간복잡도가 $O(N^2)$ 에서 $O(N)$ 으로 드라마틱하게 줄어들기 때문입니다.

101. 정렬 후 탐색 $O(n \log n)$ vs 이중 반복문 $O(n^2)$ — 데이터가 10만 건일 때 어느 쪽을 선택하는가?

답안: `**정렬 후 탐색( $O(N \log N)$ )`입니다. 10만 건 기준 $O(N^2)$ 은 100억 번 연산으로 시간 초과지만, $O(N \log N)$ 은 약 170만 번 내외로 충분히 통과합니다.

102. HashMap의 거의 모든 메서드가 $O(1)$ 인 이유를 해시 구조와 연결지어 설명하라

답안: 키를 해시 함수에 넣으면 배열의 인덱스(주소값)가 바로 나오기 때문에, 데이터 양과 상관없이 한 번에 접근이 가능하기 때문입니다.

103. `containsValue()`만 $O(n)$ 인 이유는?

답안: 키와 달리 값(value)은 해시화되어 저장되지 않으므로, 특정 값이 있는지 찾으려면 리스트를 처음부터 끝까지 훑는 것과 같은 원리로 동작하기 때문입니다.

104. `keySet()`, `values()`, `entrySet()`이 $O(1)$ 인 이유는? (뷰 패턴)

답안: 실제로 새 컬렉션을 만드는 게 아니라, `**원본 Map을 바라보는 창문(View)**`만 제공하기 때문입니다. 호출 시점에는 데이터 복사가 일어나지 않습니다.

105. 코딩테스트에서 시간 초과가 날 때 가장 먼저 의심해야 할 것은? (자료구조 선택 관점)

답안: `**"반복문 안에서  $O(N)$ 짜리 메서드를 호출하고 있지 않은가?"`를 가장 먼저 의심해야 합니다. (예: `ArrayList.contains`, `indexOf`, `remove` 등)

106. Java ArrayList에서 `remove(2)`가 인덱스 제거인지 값 제거인지 어떻게 구분하는가?

답안: 매개변수가 `int`면 인덱스, `Integer` 객체면 값으로 인식합니다. 숫자가 담긴 리스트에서 값 '2'를 지우고 싶다면 `remove(Integer.valueOf(2))`라고 명시해야 합니다.

107. JS에서 `[10, 9, 1, 2].sort()`의 결과는? 왜 이렇게 되는가?

답안: `[1, 10, 2, 9]`입니다. 숫자를 문자열로 보고 앞글자부터 비교하기 때문입니다. 반드시 `(a, b) => a - b` 콜백을 줘야 합니다.

108. Java subList의 반환값이 뷰라는 건 무슨 의미인가? 뷰를 수정하면 어떻게 되는가?

답안: 원본의 일부를 슬라이스해서 보여주는 '창문'이라는 뜻입니다. 뷰를 수정하면 원본도 바뀌고, 원본의 구조가 바뀌면 뷰를 사용할 때 에러(`ConcurrentModificationException`)가 날 수 있습니다.

109. JS에서 `splice`와 `slice`를 혼동하면 어떤 문제가 생기는가?

답안: `slice`를 썼는데 원본이 바뀔 거라고 착각하거나, `splice`를 썼는데 원본이 유지될 거라고 생각하면 로직이 완전히 꼬이게 됩니다. (원본 파괴 여부가 핵심)

110. JS 2차원 배열을 `new Array(3).fill(new Array(3).fill(0))`로 만들면 어떤 문제가 생기는가?

답안: 모든 행이 동일한 배열 객체 참조를 공유하게 됩니다. 즉, `arr[0][0] = 1`을 하면 모든 행의 0번 인덱스가 1로 바뀌는 대참사가 일어납니다.

112. Java의 HashMap에 대응하는 JS 자료구조는? Object와 Map 중 코테에서 주로 쓰는 건?

답안: JS의 Map 또는 Object입니다. 코테에서는 메서드가 더 풍부하고 키 순서가 보장되는 Map을 주로 권장합니다.

115. Java에는 있지만 JS에는 없는 메서드, 또는 그 반대를 하나씩 말해라

답안:

- Java에만: `PriorityQueue`(우선순위 큐)가 기본 제공되지만 JS는 직접 구현해야 합니다.
- JS에만: 배열의 `map`, `filter` 등을 스트림 개방 없이 배열에서 바로 체이닝할 수 있어 문법이 더 간결합니다.